

🔧 COMPLETE TECH STACK - ZERO FLAWS BLUEPRINT

FIRST: Understand What You're Building

text

Your system has 5 parts:

- Part 1: Download + Read satellite images
- Part 2: AI that understands those images
- Part 3: Agent brain that routes questions
- Part 4: Backend server
- Part 5: Frontend website

Each part needs specific tools.

Here is EXACTLY what to use.

PART 1: Satellite Image Handling

What you need to read satellite files:

text

LIBRARY 1: Rasterio

What it does: Opens GeoTIFF satellite files

Why this: Industry standard, nothing better exists

Install: pip install rasterio

LIBRARY 2: GDAL

What it does: Converts between satellite formats

Why this: Every satellite tool uses GDAL internally

Install: conda install gdal

Note: Install via conda NOT pip (pip breaks often)

LIBRARY 3: NumPy

What it does: Handles image as number arrays

Why this: Satellite images are just number grids

Install: pip install numpy

LIBRARY 4: sentinelsat

What it does: Downloads images from ESA automatically

Why this: Official ESA download library

Install: pip install sentinelsat

LIBRARY 5: pyproj

What it does: Handles map coordinates

Why this: Satellite images use special coordinates

Install: pip install pyproj

LIBRARY 6: OpenCV

What it does: Basic image operations

Why this: Resize, crop, convert images

Install: pip install opencv-python

PART 2: AI Models (The Brain)

Six specific models for six specific jobs:

MODEL 1: For Understanding Single Images (VQA + Captioning)

text

USE: GeoChat

What it is:

Vision Language Model already trained

specifically on satellite images

Why NOT regular BLIP-2 or LLaVA:
PS explicitly says generic VLM fails
GeoChat = already satellite-aware
Judges will test this specifically

Where to get it:
HuggingFace: MBZUAI/GeoChat
GitHub: mbzuai-oryx/GeoChat

What it can do out of box:
→ Describe satellite images
→ Answer questions about images
→ Identify land cover
→ Detect objects in satellite images

What you add on top:
Fine-tune on BigEarthNet data
to make it even more accurate

Install:
pip install transformers
pip install torch torchvision
from transformers import AutoModel
MODEL 2: For Change Detection Between Two Images
text

USE: ChangeFormer

What it is:
Transformer model that compares
two satellite images and finds differences

Why this specific model:
Best performing open source change
detection model available
Works on Sentinel imagery
Already tested on satellite data

Where to get it:
GitHub: justchenhao/ChangeFormer

What it does:
Takes image from 2020 + image from 2024
Returns pixel map of what changed
Green = no change, Red = changed

Training data to use:
LEVIR-CD dataset (free download)
WHU-CD dataset (free download)

Install:
git clone github.com/justchenhao/ChangeFormer
pip install -r requirements.txt
MODEL 3: For Answering Questions About Changes
text

USE: Fine-tuned GeoChat on CDVQA

What it is:
Same GeoChat model but trained
specifically on change question answering

CDVQA dataset contains:

USE: RemoteCLIP

What it is:

CLIP model trained on satellite images
Matches text descriptions to image regions

Why needed:

Helps agent understand what kind
of image was uploaded
Helps route to correct specialist model

Where to get:

HuggingFace: flywire/RemoteCLIP

Install:

```
from transformers import CLIPModel
(Load RemoteCLIP weights on top)
PART 3: Fine-Tuning Tools
To make models better on satellite data:
text
```

TOOL 1: PEFT (Parameter Efficient Fine-Tuning)

What it does:

Fine-tunes large models without
needing massive GPU
Uses LoRA technique

Why critical:

GeoChat is 7 billion parameters
Fine-tuning normally needs 8 GPUs
With PEFT/LoRA = works on 1 GPU
= Works on Google Colab Pro

Install: pip install peft

TOOL 2: Hugging Face Trainer

What it does:

Handles training loop automatically
Saves checkpoints automatically
Evaluates during training

Install: pip install transformers accelerate

TOOL 3: Weights and Biases (wandb)

What it does:

Tracks your training progress
Shows graphs of accuracy improving
Free for students

Install: pip install wandb

TOOL 4: Google Colab Pro+

What it is:

Cloud GPU for training
A100 GPU available
Cost: ~₹2000/month

Why:

Your laptop GPU not enough

for fine-tuning 7B model
Colab Pro+ = sufficient
PART 4: Agent Orchestration
The brain that connects everything:
text

FRAMEWORK: LangGraph

What it is:

Framework for building AI agents
that can make decisions and
call different tools/models

Why LangGraph not LangChain:

LangGraph = for complex multi-step agents

LangChain = simpler single-step chains

Your system needs complex routing

= LangGraph is correct choice

Install: `pip install langgraph langchain`

What your agent does:

Step 1: Receives user query + images

Step 2: Classifies query type
(VQA? Change? SAR? Grounding?)

Step 3: Checks image type
(Single? Bi-temporal? Cross-modal?)

Step 4: Selects correct models

Step 5: Runs models in correct order

Step 6: Combines outputs

Step 7: Generates audit log

Step 8: Returns final answer

Agent Structure:

QUERY CLASSIFIER NODE

→ What is the user asking?

→ Options: VQA, Caption, Change,
Grounding, SAR-Fusion

IMAGE VALIDATOR NODE

→ How many images uploaded?

→ What type? (Single/Pair/Cross-modal)

→ What format? (GeoTIFF/PNG/JPEG)

→ Are they compatible?

MODEL ROUTER NODE

→ Based on query + image type

→ Selects which models to run

→ Defines execution order

EXECUTION NODE

→ Runs selected models

→ Passes outputs between models

→ Handles errors gracefully

OUTPUT COMBINER NODE

→ Merges text + visual outputs

→ Calculates confidence score

→ Generates execution summary

AUDIT LOGGER NODE

- Records what ran
- Records parameters used
- Records time taken
- This is what ISRO evaluates

PART 5: Backend Server
text

FRAMEWORK: FastAPI

What it is:
Python web server framework
Fastest Python backend available

Why FastAPI not Flask:
FastAPI = async + faster + auto documentation
Flask = older, slower for AI workloads

Install: `pip install fastapi uvicorn`

YOUR API ENDPOINTS:

POST /upload
→ User uploads satellite images
→ Returns image ID for reference

POST /query
→ User sends question + image IDs
→ Agent processes everything
→ Returns job ID

GET /result/{job_id}
→ Frontend polls for results
→ Returns answer when ready

GET /status/{job_id}
→ Returns current agent step
→ For real-time progress display

POST /download-image
→ Downloads satellite image from ESA
→ Given location + date
→ Returns image ready for analysis

WebSocket /ws/status
→ Real-time agent progress updates
→ Shows which model is running now

TASK QUEUE: Celery + Redis

What it does:
Runs AI models in background
Frontend stays responsive
Multiple users handled

Install: `pip install celery redis`
PART 6: Frontend Website
text

FRAMEWORK: React

What it is:
JavaScript framework for building
interactive web applications

Why React:

Fast, component-based, large community
Perfect for real-time updates

Setup: `npx create-react-app satquery-ai`

KEY LIBRARIES FOR FRONTEND:

Leaflet.js

- Shows satellite images on interactive map
- Draws bounding boxes on map
- Shows change maps overlaid on map
- Install: `npm install react-leaflet`

Axios

- Connects frontend to your backend
- Sends queries, receives results
- Install: `npm install axios`

Tailwind CSS

- Makes UI look professional quickly
- No custom CSS needed
- Install: `npm install tailwindcss`

React Query

- Handles polling for results
- Auto-refreshes when results ready
- Install: `npm install @tanstack/react-query`

Chart.js

- Shows confidence scores as charts
- Shows benchmark scores
- Install: `npm install react-chartjs-2`

YOUR PAGES:

Page 1: Upload + Query

- Image upload area (drag and drop)
- Image type selector
(Single / Bi-temporal / Cross-modal)
- Query text box
- Submit button

Page 2: Results

- Original images displayed
- AI answer in text
- Visual output on map
(change maps, bounding boxes)
- Confidence score
- Execution audit log
- Download report button

Page 3: Live Search

- Type any location in India
- System downloads satellite image
- Auto-analyzes and shows result

PART 7: Deployment
text

CLOUD: Google Cloud Platform (GCP)

Why GCP:

You already have GCP experience
Free \$300 credits for new accounts
T4 GPU available for inference

SETUP ON GCP:

VM Instance:

- n1-standard-4 (4 CPU, 15GB RAM)
- 1x NVIDIA T4 GPU
- 100GB SSD storage
- Ubuntu 20.04

Services:

- Cloud Run for backend API
- Cloud Storage for satellite images
- Cloud SQL for metadata
- Firebase for real-time updates

CONTAINERIZATION: Docker

What it does:

Packages your entire system
into one portable container
Deploy anywhere in minutes

Files needed:

Dockerfile (backend)
Dockerfile (frontend)
docker-compose.yml (runs everything)

DOMAIN: Custom URL

Get free domain: satquery-ai.vercel.app

OR buy: satqueryai.in (₹500/year)

Judges need to access live URL

PART 8: Evaluation + Benchmarks

text

TOOLS TO MEASURE YOUR AI PERFORMANCE:

For VQA Accuracy:

- pip install evaluate
- Measures how correct your answers are
 - Gives you percentage score

For Captioning Quality:

- pip install pycocoevalcap
- Measures BLEU, CIDEr, METEOR scores
 - Standard metrics ISRO will check

For Change Detection:

- Custom F1 score calculation
- How accurately changes detected
 - Precision and Recall

TARGET SCORES TO AIM FOR:

RSVQA Accuracy:	> 65% (aim for 70%)
VRSBench VQA:	> 60% (aim for 65%)
Change Detection F1:	> 0.65 (aim for 0.70)
Captioning BLEU-4:	> 20 (aim for 25)

These are COMPETITIVE scores
Not state of art but good enough
to win SIH at student level
COMPLETE STACK SUMMARY
text

CATEGORY	TOOL	WHY
Satellite Data	rasterio GDAL sentinelst pyproj numpy	Read GeoTIFF files Convert formats Download from ESA Handle coordinates Process image arrays
Vision Language	GeoChat RemoteCLIP RSVG	RS-adapted VLM Image-text matching Visual grounding
Change Detection	ChangeFormer CDVQA dataset	Bi-temporal analysis Change VQA training
SAR Processing	Custom CNN BigEarthNet	SAR feature extraction Training data (SAR+MS)
Fine-Tuning	PEFT/LoRA HF Trainer wandb Google Colab Pro	Efficient fine-tuning Training management Track experiments GPU for training
Agent Framework	LangGraph LangChain Pydantic	Agent orchestration Tools and chains Data validation
Backend	FastAPI Celery Redis uvicorn	REST API server Background tasks Task queue ASGI server
Frontend	React Leaflet.js Tailwind CSS Axios	Web interface Map visualization Styling API communication
Deployment	Docker GCP nginx	Containerization Cloud hosting Reverse proxy
Evaluation	evaluate pycocoevalcap scikit-learn	VQA metrics Captioning metrics F1 score calculation

Installation Order (Do Exactly This)
text

STEP 1: Create conda environment
conda create -n satquery python=3.10
conda activate satquery

STEP 2: Install GDAL first (always first)
conda install -c conda-forge gdal

STEP 3: Install satellite tools

```
pip install rasterio pyproj sentinelsat
```

STEP 4: Install PyTorch with CUDA

```
pip install torch torchvision --index-url  
https://download.pytorch.org/whl/cu118
```

STEP 5: Install AI libraries

```
pip install transformers accelerate peft  
pip install einops timm opencv-python
```

STEP 6: Install agent framework

```
pip install langgraph langchain
```

STEP 7: Install backend

```
pip install fastapi uvicorn celery redis
```

STEP 8: Install evaluation tools

```
pip install evaluate pycocoevalcap  
pip install scikit-learn wandb
```

STEP 9: Frontend (separate terminal)

```
npx create-react-app satquery-frontend  
cd satquery-frontend  
npm install react-leaflet leaflet  
npm install axios tailwindcss  
npm install @tanstack/react-query  
One Thing That Will Break Everything If Wrong  
text
```

CRITICAL WARNING:

GDAL installation is tricky
If done wrong NOTHING works

CORRECT WAY:

```
conda install -c conda-forge gdal
```


(Then install rasterio after)

WRONG WAY:

```
pip install gdal
```


(This breaks 90% of the time)

If GDAL is installed correctly
everything else will work fine
If GDAL breaks
your entire satellite processing fails

Do this first
Test it works
Then install everything else
Bottom Line
text

You are building:

1. A file reader for satellite images
(rasterio + GDAL)
2. Six AI models that understand
those satellite images
(GeoChat + ChangeFormer + others)
3. A smart agent that decides
which model to use

(LangGraph - your strength)

4. A server that handles requests
(FastAPI)
5. A website for users
(React + Leaflet maps)
6. Everything deployed on cloud
(GCP + Docker)

Nothing here is impossible.
Each piece is well documented.
Each piece has tutorials available.
You just need to connect them all.

That connection = your agent.
Agent = your strongest skill.
Start there.
Build outward.